L1 和 L2 正则化

学习入门笔记：从概念原理到工程实践

基于 CS231N • 机器学习基础 • L1 和 L2 正则化

核心定义：Norm 用来度量参数的大小。一旦写进损失函数，它就不再是纯数学概念，而是对模型的一条约束：**优化器会倾向于选择权重较小、更稳定的解，而不是极端化的参数。**

三个关键概念：

- **Norm** 是度量：L1 看绝对值之和，L2 看平方和。
- **Regularization** 是策略：把某种“偏好”写进目标函数。
- **L1 / L2 penalty** 是两种常见实现：L1 更容易压出稀疏解，L2 更像整体收缩。

重要区分：本部分讨论的是 **目标函数中的 norm penalty**（参数惩罚），与 BatchNorm / LayerNorm 等归一化层是完全不同的概念。

核心机制 目标函数与参数惩罚

最常见的写法

$$\|w\|_1 = \sum_{i=1}^{d} |w_i|, \qquad \|w\|_2^2 = \sum_{i=1}^{d} w_i^2$$

$$J(w) = \mathcal{L}(w) + \lambda \|w\|_1 \qquad \text{或} \qquad J(w) = \mathcal{L}(w) + \frac{\lambda}{2}\|w\|_2^2$$

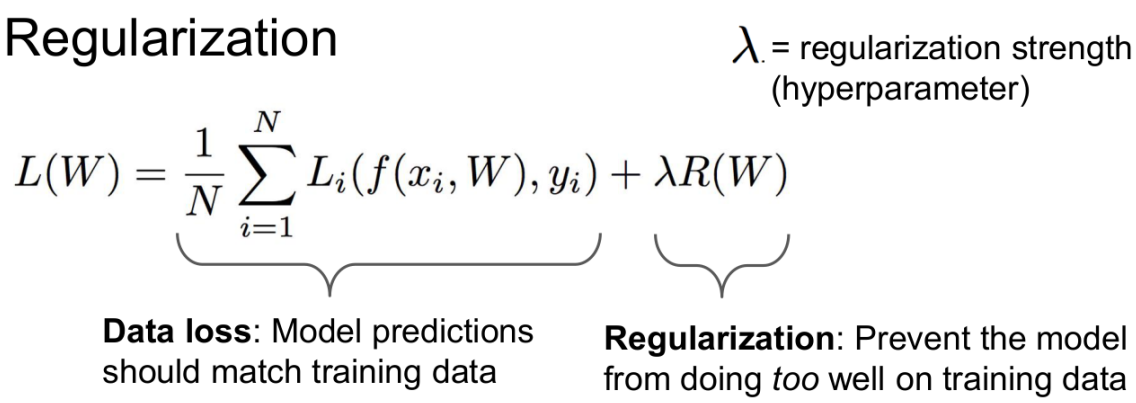
其中 $\mathcal{L}(w)$ 是数据损失，λ 是正则强度。λ 越大，模型越倾向于选择参数更小、更平滑的解。

优化器实际在做什么：

1. 先计算任务误差（数据损失）。
2. 再计算参数规模惩罚（正则项）。
3. 将两者相加为总损失，再一起反向传播更新参数。

关键点：regularization 从训练第一步就生效，不是训练结束后的“补丁”。

损失函数表达式



Simple examples
L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$
L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$
Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

Fei-Fei Li, Ehsan Adeli, Zane Durante Lecture 3 - 26 April 9, 2024

一句话直觉：norm 本来只是“量大小”，写进 loss 以后就变成“告诉模型别太放飞”的规则。

梯度视角 L1 和 L2 的参数更新差异

从目标函数到梯度变化:

当损失函数加入 penalty 后，梯度总表达式如下。它表示每次更新都受到两股力共同影响：一股追求拟合数据，一股限制参数过大。

梯度总表达式

$$\nabla_w J(w) = \nabla_w \mathcal{L}(w) + \nabla_w\big(\lambda\,\Omega(w)\big)$$

其中 $\Omega(w)$ 表示正则项（如 $\|w\|_1$ 或 $\frac{1}{2}\|w\|_2^2$）。当写成 $\frac{\lambda}{2}\|w\|_2^2$ 时，其梯度正好是 λw，这是工程里常见写法。

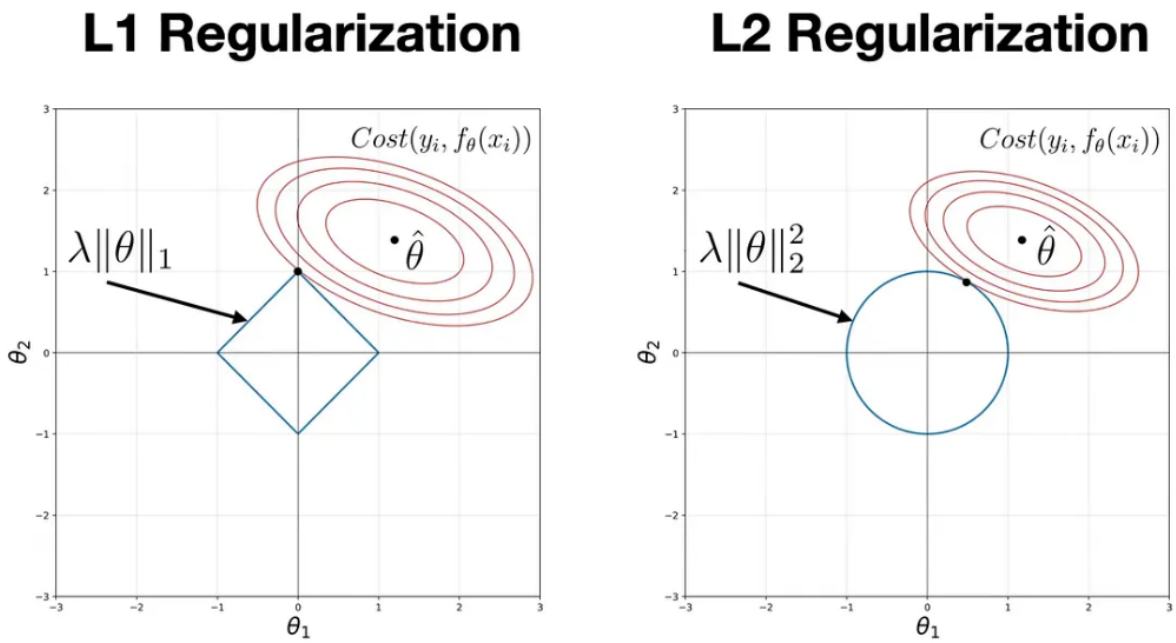
L1 和 L2 的更新风格:

- **L2：** $\nabla\frac{\lambda}{2}\|w\|_2^2 = \lambda w$。参数越大，回拉越强，整体表现为平滑收缩。
- **L1：** $\nabla\lambda\|w\|_1 = \lambda\cdot\mathrm{sign}(w)$（在 $w=0$ 处采用次梯度，$\partial|w| \in [-1,1]$）。每个维度受到近似恒定压力，小参数更容易被压到 0。
- **结果差异：**L2 常得到“都变小但仍保留”的解，L1 更容易得到稀疏解。

几何直觉：二维空间里 L1 边界是菱形，L2 边界是圆形

Geometric Intuition: Why shape changes everything

To really understand the difference between L1 and L2, it helps to visualize optimization in two dimensions, say $\theta 1$ and $\theta 2$. See the red ellipses representing contours of the loss function. These ellipses show all parameter values that produce the same loss. Without regularization, we simply move toward the center of the ellipses (the minimum loss point).



几何约束形式

$$\min_w \mathcal{L}(w) \quad \text{s.t.} \quad \Omega(w) \le c$$

$$\text{L1: } |w_1| + |w_2| \le c \qquad \text{L2: } w_1^2 + w_2^2 \le c$$

为什么 L1 更容易得到稀疏解:

L1 边界是菱形，L2 边界是圆形。结合上图可直观看到：红色椭圆表示损失函数等高线，越靠中心损失越小；蓝色边界表示正则化约束下参数允许取值的区域。优化时，可以理解为等高线不断向内收缩，直到第一次与约束边界相切，该切点就是最终解。由于 L1 的约束区域是带尖角的菱形，且尖角常位于坐标轴上，最优解更容易出现在某些参数恰好为 0 的位置，从而产生稀疏解；而 L2 的约束区域是光滑圆形，更倾向于让所有参数整体缩小，而不是直接压成 0。

一句话总结：L1 更偏向改变参数的“结构”（把部分维度压到 0），L2 更偏向改变参数的“尺度”（整体收缩）。

L2 正则化 为什么它是默认选择

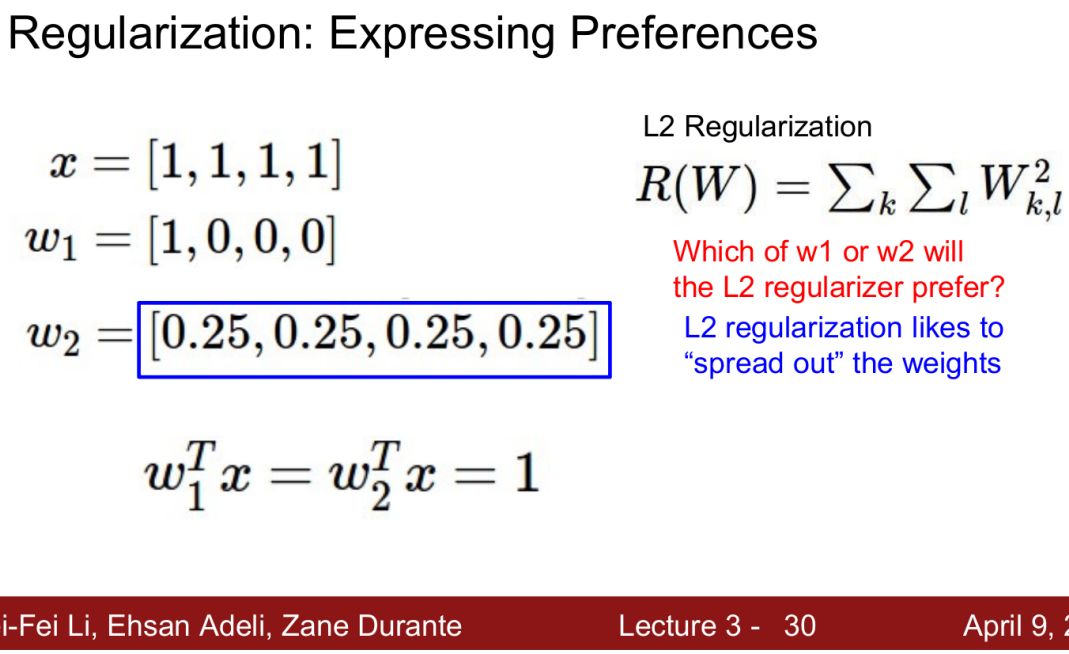
L2 常被默认采用的原因:

L2 的特点是按参数大小连续收缩：权重越大，惩罚越明显；权重越小，惩罚更温和。它通常不会把参数直接压成 0，因此训练轨迹更平滑，在多数任务上是稳妥起点。

L2 的典型使用场景:

- **线性模型/逻辑回归：**常作为默认基线，调参相对简单。
- **深度神经网络：**常通过 weight decay 来实现类似 L2 的偏好。
- **高维连续特征任务：**希望保留大部分特征时，L2 往往更稳。

通俗表达：L2 更像“整体收紧”，而不是“点名删除某个特征”。



对比分析 L1 与 L2 的不同偏好

公式背后是参数偏好不同:

L1 与 L2 的关键差别不在“公式长什么样”，而在“它们希望参数变成什么样”。L1 倾向稀疏解（很多参数为 0），L2 倾向密集但幅度更小的解。

实用对比框架:

方法	产生的结果特征	适用场景描述
L1	大量参数可被压到 0	需要特征筛选、可解释性高、模型稀疏
L2	参数整体缩小但多数保留	追求训练稳定、泛化稳健、默认基线

偏差-方差权衡角度

Bias-Variance Tradeoff

Every ML model's error can be decomposed into three parts:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

High Bias (Underfitting)

Model is too simple. Can't capture the true pattern.

Example: Fitting a line to parabolic data.

High Variance (Overfitting)

Model is too complex. Fits noise in training data.

Example: 20-degree polynomial through 10 points.

Regularization increases bias slightly but dramatically reduces variance. The tradeoff is usually worth it.

Bias–Variance Tradeoff:

- **bias 高：**模型过于简单，容易欠拟合。
- **variance 高：**模型对训练数据过于敏感，容易过拟合。
- **regularization：**通常以“略增 bias、显著降 variance”的方式改善泛化。

Elastic Net 何时需要 L1 和 L2 混合

为什么要混合：

单独使用 L1 时，强相关特征之间可能出现不稳定“轮换”；单独使用 L2 时，又可能删不掉明显无用特征。Elastic Net 结合两者，目标是在稀疏性和稳定性之间取得平衡。

Elastic Net 的标准形式

$$J(w) = \mathcal{L}(w) + \lambda_1 \|w\|_1 + \frac{\lambda_2}{2} \|w\|_2^2$$

Elastic Net 的参数化形式

$$J(w) = \mathcal{L}(w) + \alpha \left[\rho \|w\|_1 + \frac{1-\rho}{2} \|w\|_2^2 \right]$$

何时优先考虑 Elastic Net:

- **高维稀疏任务：**如文本、广告、推荐等“特征远多于样本”场景。
- **强相关特征并存：**希望避免 L1 单独使用导致的解不稳定。
- **兼顾解释与稳健：**既想筛特征，又不想训练过程太抖动。

稳定性分析 为什么较小权重更稳定

小权重通常更稳的三个原因:

1. **对输入扰动不敏感**：输入有轻微波动时，输出不容易大幅跳变。
2. **不易放大噪声**：模型更少依赖极端系数去“硬记训练细节”。
3. **优化过程更平滑**：参数更新更均匀，训练更容易稳定收敛。

核心结论：regularization 不保证绝对最优，但它会显著提高得到“可泛化解”的概率。

理解 λ 的三种取值:

- λ 太小：约束过弱，过拟合风险较高。
- λ 适中：训练拟合与验证表现通常更平衡。
- λ 太大：约束过强，模型易欠拟合（高偏差）。

调参直觉：λ 变大不是单纯“更强更好”，而是通常对应 variance 下降、bias 上升。

实践中调整 λ 的标准流程:

1. **先做对数网格搜索**：例如 $[1e^{-5}, 1e^{-4}, 1e^{-3}, 1e^{-2}, 1e^{-1}]$。
2. **重点看训练/验证曲线组合**：
   - 训练降、验证升 $\rightarrow$ 过拟合，适当增大 λ。
   - 训练和验证都高且降不动 $\rightarrow$ 约束可能过强，减小 λ。
   - 两者都稳步下降且间距可控 $\rightarrow$ 当前设置较合理。
3. **再做局部细调**：在最佳点附近缩小步长继续搜索。

重点记忆 一页记住核心概念

五条必记结论：

- **定义：**L1/L2 都是加在目标函数里的参数惩罚项。
- **机制：**正则化改变总损失，因此会改变梯度更新方向。
- **差异：**L1 偏稀疏（更容易得到 0 权重），L2 偏平滑收缩。
- **应用：**L2 常作默认基线；L1 更适合强调特征筛选的任务。
- **边界：**λ 过大易欠拟合，过小易过拟合，需看验证集调参。

重点记忆模板（直接背这 5 句）：

1. **先定义：**L1/L2 是加在目标函数里的参数惩罚项。
2. **讲机制：**它们改变了总损失，也就改变了梯度更新路径。
3. **讲差异：**L1 偏稀疏，L2 偏平滑收缩。
4. **讲应用：**L2 常作默认基线，L1 更常用于特征筛选需求强的场景。
5. **讲边界：**正则过强会欠拟合，需依赖验证集调 λ。

易混点速记

- 误区 1：把 norm 惩罚与 BatchNorm / LayerNorm 混为一谈。
- 误区 2：只会背公式，解释不了优化路径为何变化。
- 误区 3：把“L2 更常见”理解成“L2 永远最优”。
- 结论提示：模型已处于高偏差时，继续增大正则通常只会更差。

实战补充：Weight Decay 与 L2 的工程区别两者目标相近，都是抑制权重过大，但实现路径有差异：

- **L2 regularization：**把惩罚项显式写进损失函数后统一求梯度。
- **Weight decay（SGD）：**在参数更新时直接衰减，效果常与 L2 接近。
- **Weight decay（AdamW）：**将衰减与自适应梯度解耦，实践中通常更稳。

实战结论：SGD 下二者常近似等价；但在 Adam 系优化器中两者并不严格等价，通常更推荐 AdamW 风格的 `weight_decay`。

代码实现 PyTorch 的落地

读代码时先看这三件事:

第一，惩罚项在哪里算；第二，总损失在哪里合并；第三，验证阶段为什么不加惩罚项。

实现时的核心思路:

- 输入：模型、样本、标签、优化器与正则强度 λ。
- 训练：先算数据损失，再遍历参数累加 L2 惩罚。
- 设定：本实验的 L2 组使用 AdamW 的 `weight_decay`（decoupled weight decay）实现收缩偏好。
- 输出：使用总损失反传，让模型同时学会拟合与克制。

PyTorch：完整的实验对比框架

```python
import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

# ==========================================
# 1. 固定随机种子
# ==========================================
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# ==========================================
# 2. 超参数
# ==========================================
TRAIN_SAMPLES = 1000
BATCH_SIZE = 64
```

```
EPOCHS = 40
LR = 1e-3
```

```python
# 这两个值是经验上比较容易拉开差距的一组
L1_LAMBDA = 1e-5
L2_WEIGHT_DECAY = 1e-4

# 如果差异还不明显，可以试:
# L1_LAMBDA = 5e-5
# L2_WEIGHT_DECAY = 5e-4

# ========================================
# 3. 数据集
# ========================================
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

full_train_dataset = datasets.FashionMNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.FashionMNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

# 只取少量训练样本，制造更强的过拟合环境
indices = np.random.permutation(len(full_train_dataset))[:TRAIN_SAMPLES]
train_dataset = Subset(full_train_dataset, indices)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Train subset size:", len(train_dataset))
print("Test size:", len(test_dataset))

# ========================================
# 4. 大 MLP 模型
```

```python
# ==========================================
class BigMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 1024),
            nn.ReLU(),
            nn.Linear(1024, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        return self.net(x)

print("\nModel structure:\n")
print(BigMLP())

# ==========================================
# 5. L1 regularization
# ==========================================
def l1_regularization(model):
    reg = torch.tensor(0.0, device=device)
    for name, param in model.named_parameters():
        # 只对 weight 做 L1，通常不对 bias 做稀疏约束
        if "weight" in name:
            reg = reg + torch.sum(torch.abs(param))
    return reg

# ==========================================
# 6. 评估函数
# ==========================================
def evaluate(model, dataloader, criterion):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
```

```
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            total_correct += (preds == labels).sum().item()
            total_samples += labels.size(0)

    # 返回纯任务损失与准确率（不包含正则项），便于比较泛化
    return total_loss / total_samples, total_correct / total_samples

# ==========================================
# 7. 获取权重统计
# ==========================================
def get_all_weights(model):
    weights = []
    for name, param in model.named_parameters():
        if "weight" in name:
            weights.extend(param.detach().cpu().numpy().ravel())
    return np.array(weights)

# ==========================================
# 8. 训练函数
# ==========================================
def train_model(reg_type="none", l1_lambda=0.0, l2_weight_decay=0.0):
    model = BigMLP().to(device)
    criterion = nn.CrossEntropyLoss()

    if reg_type == "l2":
        # L2 组采用 AdamW 的 decoupled weight decay
        optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=
            l2_weight_decay)
    else:
        optimizer = optim.Adam(model.parameters(), lr=LR)

    history = {
        "train_loss": [],
        "train_acc": [],
        "test_loss": [],
        "test_acc": [],
        "gap": []
    }
```

```python

    for epoch in range(EPOCHS):
        model.train()
        running_loss = 0.0
        total_correct = 0
        total_samples = 0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)

            # 基础任务损失（分类损失）
            loss = criterion(outputs, labels)

            if reg_type == "l1":
                # L1 采用显式加项: loss = data_loss + lambda * ||w||_1
                loss = loss + l1_lambda * l1_regularization(model)

            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            total_correct += (preds == labels).sum().item()
            total_samples += labels.size(0)

        train_loss = running_loss / total_samples
        train_acc = total_correct / total_samples

        # 注意: test loss 这里看纯分类损失, 不加正则项, 更方便比较泛化
        test_loss, test_acc = evaluate(model, test_loader, criterion)

        # 泛化差距: 越大通常表示过拟合越明显
        gap = train_acc - test_acc

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)
        history["gap"].append(gap)

```

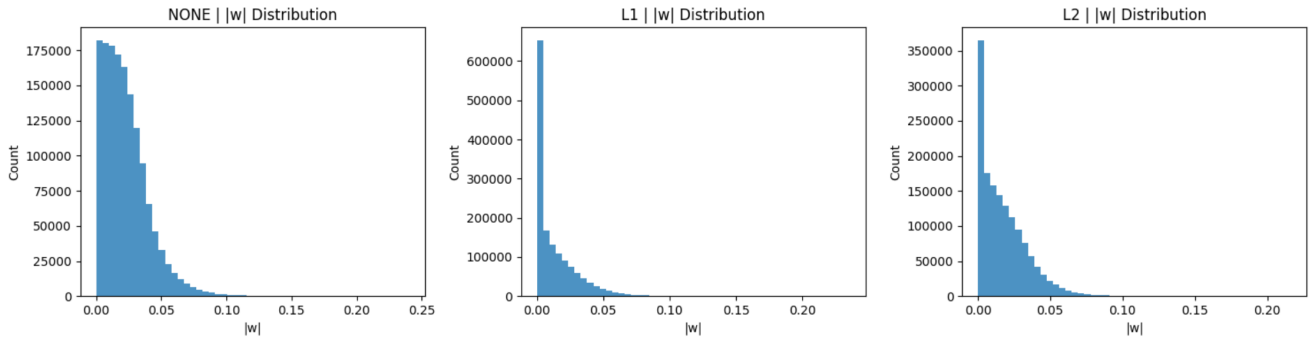
```python
        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(
                f"[{reg_type.upper():^4}] "
                f"Epoch {epoch+1:02d}/{EPOCHS} | "
                f"Train Loss: {train_loss:.4f} | "
                f"Train Acc: {train_acc:.4f} | "
                f"Test Loss: {test_loss:.4f} | "
                f"Test Acc: {test_acc:.4f} | "
                f"Gap: {gap:.4f}"
            )

    return model, history

# ==========================================
# 9. 训练三组模型
# ==========================================
results = {}

# 三组设置：无正则、显式 L1、AdamW weight decay（L2 收缩偏好）
configs = {
    "none": {"l1_lambda": 0.0, "l2_weight_decay": 0.0},
    "l1":   {"l1_lambda": L1_LAMBDA, "l2_weight_decay": 0.0},
    "l2":   {"l1_lambda": 0.0, "l2_weight_decay": L2_WEIGHT_DECAY},
}

for reg_type in ["none", "l1", "l2"]:
    print(f"\n========== Training {reg_type.upper()} ==========")
    model, history = train_model(
        reg_type=reg_type,
        l1_lambda=configs[reg_type]["l1_lambda"],
        l2_weight_decay=configs[reg_type]["l2_weight_decay"]
    )
    results[reg_type] = {
        "model": model,
        "history": history
    }

# ==========================================
# 10. 画 Loss 曲线
# ==========================================
plt.figure(figsize=(10, 6))
for reg, label in zip(["none", "l1", "l2"], ["No Reg", "L1", "L2"]):
    plt.plot(results[reg]["history"]["train_loss"], label=f"{label} Train
        Loss")
```

```
    plt.plot(results[reg]["history"]["test_loss"], linestyle="--", label=f"{label} Test Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Train/Test Loss Comparison")
plt.legend()
plt.grid(True)
plt.show()

# ========================================
# 11. 画 Accuracy 曲线
# ========================================
plt.figure(figsize=(10, 6))
for reg, label in zip(["none", "l1", "l2"], ["No Reg", "L1", "L2"]):
    plt.plot(results[reg]["history"]["train_acc"], label=f"{label} Train Acc")
    plt.plot(results[reg]["history"]["test_acc"], linestyle="--", label=f"{label} Test Acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Train/Test Accuracy Comparison")
plt.legend()
plt.grid(True)
plt.show()

# ========================================
# 12. 画 Generalization Gap
# ========================================
plt.figure(figsize=(10, 6))
for reg, label in zip(["none", "l1", "l2"], ["No Reg", "L1", "L2"]):
    plt.plot(results[reg]["history"]["gap"], label=f"{label} Gap")
plt.xlabel("Epoch")
plt.ylabel("Train Acc - Test Acc")
plt.title("Generalization Gap")
plt.legend()
plt.grid(True)
plt.show()

# ========================================
# 13. 权重分布图
# ========================================
plt.figure(figsize=(15, 4))
for i, reg in enumerate(["none", "l1", "l2"], 1):
    plt.subplot(1, 3, i)
```

```
    w = get_all_weights(results[reg]["model"])
    plt.hist(np.abs(w), bins=50, alpha=0.8)
    plt.title(f"{reg.upper()} | |w| Distribution")
    plt.xlabel("|w|")
    plt.ylabel("Count")
plt.tight_layout()
plt.show()

# ========================================
# 14. 最终结果汇总
# ========================================
print("\nFinal Summary")
print("-" * 90)

for reg in ["none", "l1", "l2"]:
    hist = results[reg]["history"]
    model = results[reg]["model"]
    w = get_all_weights(model)

    final_train_acc = hist["train_acc"][-1]
    final_test_acc = hist["test_acc"][-1]
    final_gap = hist["gap"][-1]
    final_test_loss = hist["test_loss"][-1]

    # 统计接近 0 的权重比例，用于观察稀疏性
    near_zero_ratio = (np.abs(w) < 1e-3).mean()
    mean_abs_w = np.mean(np.abs(w))

    print(
        f"{reg.upper():>4} | "
        f"Train Acc: {final_train_acc:.4f} | "
        f"Test Acc: {final_test_acc:.4f} | "
        f"Test Loss: {final_test_loss:.4f} | "
        f"Gap: {final_gap:.4f} | "
        f"Mean |w|: {mean_abs_w:.6f} | "
        f"Near Zero Ratio: {near_zero_ratio:.4f}"
    )
```

实验结果输出（Final Summary）

```
Model  | Train Acc  | Test Acc  | Test Loss  | Gap     | Mean |w|   | Near-Zero
   Ratio
------+-----------+----------+-----------+--------+-----------+----------------

NONE   | 1.0000     | 0.8053    | 1.3417     | 0.1947 | 0.022735   | 0.0259
L1     | 0.9940     | 0.8067    | 1.0837     | 0.1873 | 0.013119   | 0.3074
L2     | 1.0000     | 0.8073    | 1.2343     | 0.1927 | 0.017547   | 0.1392
```

解读这组结果:

- **泛化表现**：在该实验设置下，L1 与 L2 的 Test Acc 都略高于无正则（NONE），说明本次实验中正则化改善了泛化表现。
- **稀疏性**：L1 的 Near-Zero Ratio 最高（0.3074），符合“L1 更容易产生稀疏解”的理论预期。
- **权重收缩**：L1/L2 的 Mean $|w|$ 都小于 NONE，说明两者都在抑制参数过大。